Hope Beacon: The Complete Handbook

Everything needed to run Open Hope Beacon, move it to a new project, and keep it working. Written for the people who run a church, and for the AI tools that will be asked to continue the work.

Version: 26 August 2026 (evening) · **Applies to:** migrations through `0040`, invite function `v22` · **Licence:** AGPL-3.0 · **Source:** `github.com/Klydo131/Open-Hope-Beacon`

> **NOTE** · How to read this
>
> **Running a church?** Parts 1 to 5 are yours. They assume no technical knowledge and nothing needs installing.
>
> **Setting the app up, or moving it?** Parts 6 to 10. Follow them in order; each one checks the one before it.
>
> **An AI tool picking this up?** Part 11 is written for you and states the invariants you must not break.

1. What the app is

Hope Beacon is a discipleship app for one church. A member of that church walks alongside one other person at a time, and the app carries what that takes: the conversation, the readings, the prayer requests, and a quiet record of how far along the journey somebody has come.

Three things define it, and every decision in the rest of this handbook comes back to one of them.

It is invitation only

There is no public sign-up and there never will be. Somebody at the church enters a name and an email address, the app sends an invitation, and that is the only door. A stranger who finds the web address sees a sign-in screen and a tutorial, and nothing else.

A conversation belongs to two people

What an Explorer says to their Guide is readable by those two and nobody else. Not other Guides, not Directors, not the person who owns the server. The only exception is a safeguarding report, which a Director may read in place, and the app says so plainly on the screen where a report is made.

The limit is people, not computers

A Guide walks with at most five Explorers at once, and the database enforces it rather than the screen. The app will run a church of a hundred on a free plan without complaint. What it cannot do is find you a sixth Guide. Growth here means recruiting and training people, and the app is built to keep that constraint visible rather than hide it behind a number that keeps rising.

2. The four roles

A person's role is chosen when they are approved, and it decides everything they can see for as long as they are in the church. **Nobody can change their own role, including the Executive Director.** That is enforced in the database, not in the app.

Role	What they do	What they can see
Explorer	Walks the journey. Reads what their Guide sends, talks with them, asks for prayer.	Their own journey, and their conversation with their Guide. Nothing about anybody else.
Guide	Walks with up to five Explorers. Chooses what to share and when. Recommends new people, but cannot invite them.	Only the Explorers paired with them. Never another Guide's people.
Director	Runs the church. Invites, approves, pairs, and reads safeguarding reports.	Everyone in their church and the counts behind them. Not private conversations, except inside a report.
Executive Director	Oversees one or more churches, and appoints Directors.	Everything a Director sees, across every church they oversee.

> **NOTE** · The Head Executive Director
>
> One account is the root of authority. It cannot be suspended or removed by anybody, including itself, so a church can never lock itself out of its own app. Guard the password for that account the way you would guard the keys to the building.

3. The journey

Five stages, and an Explorer moves through them at their own pace. The stage is a note for the Guide, not a score, and nothing in the app hurries anybody along.

Stage	What it means
Beginner	Just beginning. Somebody has said yes to being walked with.
Connect	Building rapport. Getting to know each other.
Care	Walking alongside. The longest stage, and usually the most valuable.
Call	A point of decision.
Cultivate	Growing in faith after that decision.

The first stage was called *Create* until August 2026. That named what the church was doing; *Beginner* names where the Explorer is, which is whose journey it is. Nothing changed in the database, so no history moved.

A sixth idea sits behind these: **Commission**. An Explorer who has been walked with becomes a Guide, and walks with somebody else. That is the whole point of the design, and it is the only kind of growth that does not run out.

4. Running it, week to week

Inviting somebody

Step 1. Sign in as a Director, open **Admin**, then the **Approvals** room.

Step 2. Enter their name and email address, choose the role, and press **Send invitation**.

Step 3. They receive an email written for that role. An Explorer, a Guide and a Director each get a different message, because each is being asked for something different.

Step 4. They choose a password. That is what finishes the sign-up.

Step 5. They appear under **Awaiting approval**. Approve them, and they can enter.

> **CAUTION** · One live invitation per person
>
> Sending a second invitation to the same address switches off the first. If somebody says the link does not work, ask whether they have two emails, and tell them to use the newest one. Never open somebody else's invitation link yourself: it works once, and opening it signs you out and starts their sign-up on your device.

Pairing a Guide with an Explorer

Open the **Pairings** room, choose one of each, and press **Create pairing**. They can talk from that moment. A Guide already carrying five will not appear in the list, because the database will not allow a sixth.

Five is a ceiling, and a church can raise it. A Guide walks with five Explorers by default, enforced by the database. A congregation with more Explorers than Guides can carry may raise that limit in Church settings, up to twenty-five. Only a Director or an Executive Director can, and only for their own church: a Guide can never give themselves more people.

Raising it is a decision, not a drift. Five is the number the design is built around because it is how many people one person can actually walk with.

The one number worth watching is unpaired Explorers. An Explorer with no Guide has been invited into an app where nothing happens. Your dashboard opens on that number for exactly this reason.

Finding one person

The approved list gets long. Once it passes five accounts a search box appears above it: type any part of a name and the list narrows as you type, showing "4 of 37" so a short list is never mistaken for a lost account.

Past six accounts the list gets a scroll bar of its own rather than growing down the page. Forty accounts is roughly four thousand pixels, and every control above the list, the search box and the bulk buttons, used to scroll out of sight the moment somebody started reading names. Now the roll and the things you do to it stay on screen together.

The "New" mark

Anybody who finished signing up in the last seven days carries a green **New** badge on the rosters and on a Guide's cards. It works out the answer from the date every time it draws, so it can never be left on somebody by mistake, and it counts from when they chose a password rather than when a Director typed their address.

The badge gives a soft ring twice when the screen draws, then stops. A mark that pulses forever is a mark people stop seeing, and then it is only noise on somebody's name. Every role gets the same badge, which is deliberate: a different mark for a new Explorer would tell everybody who can see the list which members are Explorers, and the app takes care elsewhere not to say that.

A device set to reduce motion gets a still outline instead of the ring.

Disapprove, or delete

These are different acts and the difference matters.

	Disapprove	Delete
What happens	The account is switched off. They cannot enter, but they and their history stay.	The account, its messages and its pairings are removed for good.
Reversible	Yes. Approve them again.	No.
Their email	Still in use by that account.	Freed. They can be invited again as a brand new member, in any role.
Use it when	Somebody is away, or you are looking into something.	Somebody has left, or an account was created by mistake.

Delete asks a second time in the row itself rather than through a browser pop-up, because on a phone that dialog appears under the thumb that just tapped Delete and the button dismissing it is the one that agrees. It says what will go before it goes. The removal is recorded in a log that outlives the person it describes, so a church can always answer who removed whom and when.

Acting on several accounts at once

Every row in the approved list has a tick box. Tick a few, or use **Select all**, and two buttons appear: **Disapprove selected** and **Delete selected**.

Three things about it are worth knowing before you use it on twenty people.

- **Select all means what you can see.** With a search showing four of thirty-seven, it takes those four. It never quietly reaches the rows the search is hiding.
- **The confirmation names everybody.** Not "delete 12 accounts", but the twelve names, because there is no way to check afterwards and no way to undo.
- **One refusal does not undo the rest.** If the database refuses one person, for instance a Director you may not act on, everybody else is still done and you are told which one failed and why.

> **NOTE** · Who may delete whom
>
> Decided by the database, not the screen. An Executive Director may act on anyone in a church they oversee. A Director may act on Guides and Explorers only, never another Director. Nobody may act on themselves, and nobody at all may act on the Head Executive Director. If an action is refused you are told why, in a sentence.

Meetings

A Guide and an Explorer arrange a time together on the same card, and both see it. Either may propose one: a title, a date and time, online or in person, and a place that becomes a map link. The other person confirms it, and either can cancel. Nobody else in the church sees any of it.

Lesson studies

A Guide writes their own. Create a series, add studies to it, attach the handouts you already use, and publish it when it is ready. Until you publish, only you can see it. Anybody in the church can then open the series, read the studies and open the files.

Directors keep the same control over everything, which is what running the church means. A Guide may edit and delete only what they wrote.

The church noticeboard

Anybody approved in the church can write a post. Explorers, Guides, Directors and Executive Directors all have **Your blog** on their own screen. Before tonight only Guides and leaders could, and an Explorer who tried was shown an error from the database.

Three audiences, and the choice is made before publishing.

Audience	Who reads it
Everyone in the church	Every approved member of your church. This is the noticeboard.
Only the people I walk with	For a Guide, their Explorers. For an Explorer, their Guide.
Only the people I choose	The people named on it, and nobody else. It only appears when there is somebody to choose.

A post stays private until it is published, and **Make private** takes it back off without losing it.

> **CAUTION** · A church-wide post is signed
>
> Choosing **Everyone in the church** puts your name and your role on it, and the screen says so before you press publish. That is on purpose: a noticeboard where some posts are signed and others are anonymous is one nobody can hold to account. The narrower audiences follow the app's ordinary rule, which does not name an Explorer's role to people who have no reason to know it.

The noticeboard is the **first thing on Home** for every role, newest first. It draws nothing at all when nobody has published, rather than leaving an empty card at the top of the screen.

Directors and Executive Directors can delete any post in their church. That is not tidying up; it is the reason an audience open to every member is safe to have at all. A church-wide megaphone with no way to switch it off is a problem waiting for a Sabbath morning. Anybody can always delete their own.

Undoing a step

Advance stage is one tap, and taps go wrong. **Undo, step back** sits beside it and puts an Explorer back a level. It asks first, and it is recorded as a correction rather than erased, so the history stays honest. The Explorer is never shown their stage either way, so a correction is invisible to the person it is about.

Numbers a Guide can see

The screen a Guide lands on opens with their own figures: how many Explorers they have, how many have **graduated**, how many are still walking, and the breakdown by level. Above that sits whatever is waiting today, which is usually short: prayer requests, and the next meeting with a name and a day.

Graduated means reached Commission: walked the whole journey and now sent to walk with somebody else. It is the number the whole design exists to produce.

Safeguarding

Anybody can report a conversation. When they do:

- Every Director of that church is notified at once.
- A Director can read the conversation in place, with what came before and after, rather than as a single quoted line.
- **The person reported is never told.** No message, no notification, nothing they could notice.

- The reporter's name is visible to Directors, because a Director cannot support them or tell a genuine concern from a grudge without it.
- Reports are never deleted, whatever is decided.

Reading the numbers

The Church screen opens with four headline figures: Explorers, Guides, Graduated, and how many are waiting for a Guide. Under them sit two charts, each answering one question a Director actually asks.

Who is using it. One panel for Guides and one for Explorers, each showing everybody on the roll for today, this week and this month. The blue part of the bar is the people Beacon recorded doing something; the brown part is the rest.

> **CAUTION** · What "active" does and does not mean
>
> **It is not a count of visits.** Beacon does not record when somebody opens the app, so a number claiming to be visits would be invented. Active means the app recorded them doing something: sending a message, a step on a journey, arranging a meeting, or writing a post or a study.
>
> An Explorer who met their Guide for coffee and wrote nothing down is in the brown part. That is not a failure and it should not be treated as one. What is worth acting on is a whole month of brown for one person, and *Waiting for a Guide* above zero.
>
> Today will almost always look low, and that is the day, not the church. Read the week and the month.

Who is arriving. Four small charts, one for each role: Executive Directors, Directors, Guides and Explorers. Choose the period from Daily, Weekly, Monthly, Quarterly or Yearly. All four share one scale, so a tall bar means the same number of people wherever it appears.

Somebody counts as arriving on the day they finished signing up, not the day their invitation was sent. An invitation that sat unopened for three weeks would otherwise land on the wrong week.

Underneath, in the same period, is what the church decided about people: how many were let in, turned down, suspended, had a suspension lifted, and removed.

> **NOTE** · Removed and deleted are one number
>
> In Hope Beacon, removing somebody from the church deletes their account. There is no separate state where a person has been put out but still has a login. The record of the removal survives them, which is the point of keeping it.

> **NOTE** · Average and middle period
>
> Both are shown because they disagree in the case that matters. One busy week after a quiet month pulls the average up; the middle week is closer to an ordinary one.

Both tables download as a CSV that opens in Excel, Google Sheets, Numbers or LibreOffice, one under the other. For a board pack, use the browser's own print, which every phone and computer can already save as a PDF. Nobody's name is in either file.

Every one of these numbers is a count. No name, no message and no prayer appears on this screen or in the file, and the part that counts messages runs inside the database and hands back only a total, so a Director reading "eleven Guides were active" is not reading anybody's conversation.

The board report

Admin has a panel with the four numbers to read out at a board meeting, and a Print button. It names nobody. If a board member wants to know how one particular person is doing, the answer is to ask the Guide walking with them. The app will not show it.

5. Getting it onto a phone

Hope Beacon installs from the browser. There is no app store, no download, and no review process. Once installed it has its own icon, opens without an address bar, and keeps working when the signal does not.

> **IMPORTANT** · iPhone and iPad: only Safari can install
>
> Apple permits only Safari to add an app to the Home Screen. Chrome, Firefox, Edge and Opera on an iPhone cannot do it, and no amount of work on our side can change that. Neither can the browser inside Messenger, Facebook or Instagram.
>
> What we can do is make leaving take one tap instead of three steps, and that is what the app now does.

If you are not in Safari: one tap

Open the app in Chrome, Firefox, Edge, or from a link inside Messenger or Facebook, and it says which browser you are in and offers a single button: **Open this page in Safari**. Tapping it reopens the page you are on, in Safari, with nothing retyped. From there, Share and Add to Home Screen work normally.

This matters most for somebody holding an invitation. The old advice was to switch to Safari, and people did that by opening Safari and typing the address, which loses the invitation link they were on. That is the version of the bug reported as "they switch to Safari and it still does not work". The button carries the exact page across.

> **CAUTION** · Honest about the limits of this
>
> The handoff uses a URL scheme Apple has never documented. Most browsers and most in-app browsers honour it. Some refuse, and when they do, *nothing happens and nothing says why*.
>
> So the written steps stay on the screen underneath the button, always, and there is a Copy link button for the person whose browser refuses both. It has been tested with simulated iPhone browsers in an automated test. It has **not** been tested on a physical iPhone.

iPhone and iPad, by hand

1. Open the church's address **in Safari**. If you are in another browser or inside a chat app, use its ••• menu and choose **Open in Safari**.
2. Tap **Share**, the square with an arrow coming out of it.
3. Scroll down and tap **Add to Home Screen**.

4. Tap **Add**, top right.

> **CAUTION** · If somebody installed before 26 August 2026
>
> What they have is a bookmark, not an app, and it will not convert itself. Older iPhones needed a tag the framework had stopped emitting, so Add to Home Screen produced an icon that opened Safari with the address bar showing. Nothing errored, which is why it was reported as "the install does not work".
>
> The fix is deployed. Those people must **delete the old icon and add it again**.

Android

In Chrome, open the ⋮ menu and choose **Add to Home screen** or **Install app**. Most phones offer it by themselves after a few visits.

Windows, Mac and Chromebook

In Chrome or Edge, look for the install icon at the right-hand end of the address bar: a screen with a downward arrow. In Safari on a Mac, choose **File**, then **Add to Dock**.

Updates

Nobody reinstalls. When a new version ships, every open copy notices within seconds and offers to refresh. The app will not reload while a message is half written.

> **GOOD TO KNOW** · An update does not sign anybody out
>
> Signing in is stored in the browser, tied to the database project, and shipping new code does not touch it. Nor does the offline cache being rebuilt, nor the crash recovery, which clears caches only. There is a test that fails the build if that ever stops being true.
>
> **Two things do end every session, and both are in Part 7:** moving to a different database project, and changing the web address.

Something to listen to

There is one player, in two sizes, and they are two views of the same thing rather than two players.

The small one sits in the right-hand column of every room. Shut, it shows what is playing and the volume. The ⋯ at its top right opens it, and it remembers which you chose on that device. Open, it has the same three tabs as the full one.

The full one is the first thing on **My Library**, and it is the only place it appears. It used to sit on My Journey and on a Guide's workspace as well, where it was the largest card on a screen meant to be about somebody's next step.

Three tabs, in the order most people want them.

- **Vault** is your own music and video, saved on this device. There is a search box over it, because a vault worth having is a vault too long to scroll, and a **Save music or video** button that puts more in. Files stay on the device and are never uploaded.

- **Playlists** are your own, saved on the device and never uploaded: name one, then add whatever is playing to it. A playlist can mix ambience and your own recordings, so rainfall behind a sermon is one list.
- **Ambience** is **made on the device as it plays**: rainfall, distant surf, plain hush. There is no file to download, it costs no data, and it works with no signal.

Both sizes drive the same sound, so starting a track in the Library and walking back to your room keeps it playing.

What somebody listens to while they read is nobody else's business, which is why the vault and the playlists stay on the device rather than in the church's database.

Waiting

Anywhere the app is fetching something, it shows the lighthouse mark turning with a word for what it is waiting on, rather than the word "Loading" on its own or nothing at all. A screen that is still fetching and a screen that has finished and found nothing used to look identical, so people pressed the button again.

A device set to reduce motion gets the same message without the spin.

Notifications

The bell in the header holds both the list and its switch. Alerts in the app are **on by default**. Device alerts are the browser's to grant: the panel offers to ask once, and if a browser has already refused it says where to change that rather than offering a button that would do nothing.

6. Email, end to end

Two providers, deliberately, because they fail in different ways and one must not be able to take the other down.

What	Sent by	Why
Invitations	**Brevo**	Three different messages, one per role, composed in the code and kept under version control.
Password resets	**Supabase**, over Brevo's SMTP	Only the auth system can mint a recovery link, so this one cannot move.

Why not Supabase for invitations too

Supabase Auth has exactly one "Invite user" template with no way to branch on a role. The moment three roles needed three different invitations, that template could no longer do the job. There is also a hard ceiling: the built-in mailer sends **two emails an hour for the whole project**, which one Director inviting three people on a Sunday afternoon would exhaust.

Setting up Brevo

Step 1. Verify your sending domain. In Brevo, go to *Senders, Domains & Dedicated IPs* and add your domain. Brevo gives you DNS records to publish; add them at whoever sells you the domain and wait for Brevo to show the domain as authenticated.

Step 2. Create an API key. *SMTP & API → API keys → Generate a new API key*. It begins with `xkeysib-`. Copy it once; Brevo will not show it again.

Step 3. Create an SMTP key as well, on the same page. This is a different key for a different job, and the password reset needs it.

Step 4. Check the IP restriction on both. Brevo can limit a key to named IP addresses, and it is two separate switches: one for API keys, one for SMTP keys. Our server has no fixed address, so a key restricted this way is refused every time and Brevo's log shows nothing at all.

Step 5. Store the API key in Supabase, never in the website's settings. See Part 8 for exactly where.

> **CAUTION** · Two Brevo traps that cost a morning each
>
> **Not every key works with the API.** Keys created for other Brevo integrations are a different type, and the sending endpoint answers "Key not found" for them, which reads as a wrong key rather than a wrong kind. Create the key from *SMTP & API → API keys* and nowhere else.
>
> **IP restriction is on by default in some accounts.** Turning it off is a real reduction in protection, and it is the owner's call. The compensating control is that the key lives in the database where only the server can read it, and rotating it takes under a minute.

Setting up the password reset

In Supabase: *Project Settings → Authentication → SMTP Settings*. Enable custom SMTP and enter Brevo's host, port `587`, your Brevo login and the **SMTP key** as the password. Set the sender to an address on the domain you verified. This also lifts the two-an-hour ceiling for everything Supabase sends.

How an invitation is actually sent

Worth understanding, because almost every email failure has been a misunderstanding of this.

1. A Director presses Send. The request reaches a small server function, the only piece of the system holding the key that can bypass the security rules.
2. It refuses if the address already belongs to a member who has finished signing up.
3. It creates or refreshes the one invitation row for that address.
4. It mints a one-time link, composes the message for that role, and hands it to Brevo.
5. If Brevo will not take it, and only then, it produces a link the Director can pass on by hand, and says why the email did not go.

> **IMPORTANT** · The rule that broke every invitation for a week
>
> An account has **one slot** for an invitation link, not a collection. Minting a second link overwrites the first, and the first stops working immediately. The function used to mint a spare link after sending the real one, which quietly destroyed the link that had just gone into somebody's inbox. Every invitation arrived dead and the error message said the link had expired.
>
> Never mint a link after a send. Anything that calls the mint function must do it before the send, or only when the send has failed.

Changing the wording of an invitation

The three messages live in the repository at `supabase/functions/invite/email.ts`. Edit them there, in plain TypeScript, and redeploy the function. A test renders all three and checks the link appears twice, the church name is escaped, and no placeholder survives into the message.

Brevo templates are supported as an alternative but not recommended: they put the words a congregation reads behind a dashboard with no version control, and somebody must build three by hand before a single invitation can go out.

7. Moving to a new project

This is the part to read twice. Moving the app means moving three separate things, and they have different consequences for the people already using it.

What moves	Effect on people already using it
The code (a new repository, a new deploy)	None. Nobody is signed out and nothing is reinstalled.
The database (a new Supabase project)	**Everyone is signed out, and their accounts do not come with it** unless you deliberately carry them over.
The web address (a new domain)	**Everyone is signed out, and every installed icon is stranded for good** on a copy that can never update.

> **IMPORTANT** · The address is the one you cannot undo
>
> A browser identifies an installed app by its web address. Change it and every phone that installed the old one keeps a copy that can never receive another update, and no amount of work on our side reaches it. The only fix is to ask every person to delete the icon and add it again.
>
> So: decide the final address *before* more people install, or accept that one day you will send that message to everybody. There is no third option. If you do move, announce it before the switch, not after.

Why a new database signs everybody out

Being signed in is one entry stored in the browser, and its name contains the database project's own identifier. A different project means a different name, so the browser looks for the old one, finds nothing, and shows the sign-in screen. The accounts themselves live inside the old project and do not travel with the code.

Two honest ways forward. Choose deliberately.

	A. Start clean	B. Carry the accounts over
What you do	Run the migrations on the new project and invite everybody again.	Copy the database, including the authentication tables, into the new project.
Passwords	Everybody sets a new one.	Survive the move.
Signed out	Yes.	Yes, unavoidably.
Risk	Low. Nothing to go subtly wrong.	Higher. Needs direct database access and careful ordering.
Right when	A demo, a pilot, or a church small enough to re-invite in an afternoon.	A congregation with real history worth keeping.

The migration checklist

In this order. Each step is checkable, and a step that cannot be checked has not been done.

Step 1. Create the new Supabase project. Note its URL and its publishable (anon) key from *Project Settings → API*. The anon key is not a secret and ships to every browser by design; the service role key never leaves the server.

Step 2. Run every migration, in filename order, from `supabase/migrations/`. They build on one another, so the order is not optional. Migration `0001` through `0035` plus the dated one. Paste each into the SQL editor, or use the Supabase CLI.

Check: the `profiles`, `pairings`, `invites` and `app_settings` tables exist, and row level security is on for all of them.

Step 3. Deploy the invite function. Deploy `supabase/functions/invite` to the new project. It is the only piece holding the service role key, and it is what sends every invitation.

Check: the function appears in *Edge Functions* and its version is the one you just deployed, not an older one that happened to be there.

Step 4. Put the Brevo key in `app_settings`. In the SQL editor:

```
insert into app_settings (key, value) values
  ('BREVO_API_KEY',     'xkeysib-your-key-here'),
  ('BREVO_SENDER',      'hello@your-domain.org'),
  ('BREVO_SENDER_NAME', 'Your Church')
on conflict (key) do update set value = excluded.value;
```

That table has row level security on and no policy granting anybody access, so only the server can read it. It is not in the repository and never will be.

Step 5. Set the sign-in redirect. *Authentication → URL Configuration*. Set *Site URL* to your address, and add `https://your-address/join` to the redirect allow list. Get this wrong and invitations arrive but land nowhere.

Step 6. Turn on custom SMTP for password resets, as in Part 6.

Step 7. Point the website at the new project. On the host, set `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`, then **redeploy**. Changing a setting alone does nothing: these are read at build time, so a saved setting with no redeploy leaves the old project connected.

Step 8. Create the first account by hand. There is no public sign-up, which leaves the first account a chicken and egg problem. In *Authentication*, add a user with your email and a password. Then in the SQL editor find that person in `profiles`, set their role to `executive` and their approval to true.

Step 9. Prove the rules before real names go in. Sign in as somebody with the least access, an Explorer, and try to reach what they should not: another person's conversation, the member list, the admin screens. The rules are enforced by the database rather than by the screens, so this is a real test. `docs/examples/prove-the-rules.sql` does the same thing faster.

Step 10. Send one real invitation to yourself and complete it end to end: receive it, choose a password, get approved, land in the app. Only then invite anybody else.

> **GOOD TO KNOW** · What "working the same" means, concretely
>
> After step 10, all of this should be true on the new project: an invitation arrives within a minute; the three roles get three different messages; a password reset arrives; an Explorer cannot see another Explorer; a Guide cannot take a sixth Explorer; deleting an account frees the address; and the app installs on an iPhone from Safari. If any one of those is false, stop and fix it before the next step rather than after.

8. Every setting, in one place

On the website host

Name	What it is	Required
`NEXT_PUBLIC_SUPABASE_URL`	Your project's address. Public by design.	Yes
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	The publishable key. Public by design; the security rules are what protect the data.	Yes
`CANONICAL_HOST`	The address the app treats as its real home, comma separated if there is more than one. What lets the app warn somebody who installed from a temporary address.	Recommended
`NEXT_PUBLIC_VAPID_PUBLIC_KEY`	Only if you add push notifications later.	No

> **IMPORTANT** · Never
>
> The service role key must never appear on the website host, and never in anything whose name begins `NEXT_PUBLIC_`. That key bypasses every security rule in the database. It belongs in the invite function and nowhere else.

In the database, in `app_settings`

Key	What it is
`BREVO_API_KEY`	The API key. Without it, invitations fall back to Supabase and its two-an-hour ceiling.
`BREVO_SENDER`	The address invitations come from. Must be on a domain Brevo has verified.
`BREVO_SENDER_NAME`	The name people see, for example your church's name.
`SITE_URL`	Where invitation links point, if it differs from the site's own idea of itself.
`BREVO_INVITE_TEMPLATE_ID_DS`	Optional. A Brevo template for Explorers, instead of the built-in message.
`BREVO_INVITE_TEMPLATE_ID_DM`	Optional. The same, for Guides.
`BREVO_INVITE_TEMPLATE_ID_ADMIN`	Optional. The same, for Directors.
`BREVO_INVITE_TEMPLATE_ID`	Optional. One template for every role, used only when no per-role template is set.

Any of these may instead be set as a secret on the invite function; the function checks its own secrets first and the table second. The table is easier to change without a redeploy.

DNS, at whoever sells you the domain

Purpose	What to add
The website	The records your host gives you when you add the domain to the project.
Email authentication	The DKIM and SPF records Brevo gives you when you add your sending domain. Without them, invitations land in spam.
DMARC	Optional but worth it once the two above are verified.

9. The database and its rules

Every privacy promise this app makes is kept by the database, not by the screens. That distinction is the whole security design: a screen can be bypassed by anybody who opens the developer tools, and a database rule cannot.

The rules that must never break

Rule	Kept by
Nobody can change their own role.	A trigger that rejects the change, and a saved profile that always writes back the role it read.
An Explorer sees only themselves.	Row level security on every table, scoped by church and by pairing.
A Guide sees only the Explorers paired with them.	The same.
A Guide carries at most five Explorers.	A trigger that counts, because a limit across rows cannot be a constraint.
The Head Executive Director cannot be removed.	The discipline check, refused before anything happens.
A removal is always recorded.	A log written before the deletion, which outlives the person it describes.
A change to somebody's details is visible to their Guide.	An append-only table with no write policy at all, filled by a trigger.
Nothing new is readable by a signed-out visitor.	A check that fails the build if any table grants anything to anonymous.
Only an approved, unsuspended member can publish a post.	The write rule on the posts table, which also pins the author to whoever is signed in and the church to their own.
A Director may take down any post in their church.	The delete rule, scoped to churches that Director actually leads.
Counting messages never exposes one.	The counting runs inside the database and returns totals. No message, and no name, ever leaves it.

The blog error, and what it actually was

Worth writing down, because it was diagnosed wrongly twice and each wrong fix looked plausible.

Publishing a post failed with `new row violates row-level security policy`, which reads exactly like a refusal to write. It was not. The app saves a post and asks for its id back in one statement, and the database applies the **read** rule to the row it is about to hand back. The read rule called a helper that looks the post up by its id, and that helper runs on a snapshot of the database taken before the row existed. So it looked for the post, could not find it, and refused to return to the author the very row it had just accepted from them.

The fix is to answer the author's own case from the row itself rather than by looking it up: the post is yours if its author is you, which needs no lookup and is true for a row nothing can see yet.

Two things made this hard to see. The message names the write, and the same insert works perfectly if you do not ask for the id back. It was proved by doing exactly that, and by widening the write rule to allow everything and watching it fail anyway.

Deleting an account, and why it used to fail

Worth stating because it is the kind of bug that hides for months. The `profiles` table hangs off the authentication table, and deleting a profile does *not* delete the account behind it. A cascade only runs one way.

So a removed person kept a working login that resolved to nothing, and their email address could never be invited again, because the check that refuses a duplicate invitation looks at the authentication table, which still held their row. Only deleting there frees the address, and that is what the app now does everywhere a member can be removed.

Backups

Two scheduled jobs live in the repository. Both are free on a public repository, and both need their secrets set before they do anything:

- **Keep awake** pings the database daily, so a free project is never paused for inactivity.
- **Backup** takes an encrypted copy weekly.

> **IMPORTANT** · Two things about backups
>
> **A backup nobody has restored is a rumour.** Restore one into a scratch project once, before you need it. The job has never been proved by a real restore.
>
> **Never let an unencrypted dump reach the repository.** Files attached to a job on a public repository can be downloaded by anyone on the internet. Encrypt before upload, or do not produce the file.

10. When something breaks

What you see	What it usually is	What to do
"This invitation link has expired or has already been used"	A newer invitation was sent to the same address, which switches off the older one. Or somebody already opened it.	Ask them to use the newest email. If unsure, press Re-send and tell them to use only what arrives after that.
The invitation email arrives empty	A template using a field the mail system cannot resolve. It abandons the whole message and sends a blank one, and nothing anywhere reports an error.	In a Supabase template, use only `{{ .ConfirmationURL }}`. Nothing else is guaranteed to exist.
No invitation arrives at all, and Brevo's log is empty	The key was refused before a send was recorded. Almost always the IP restriction, occasionally a key of the wrong type.	Check the IP setting on *both* API and SMTP keys. Then confirm the key was made under *SMTP & API → API keys*.
"one message per address per minute"	Working as intended. A second message to one address inside a minute is held back.	Wait the number of seconds shown, then press Send once.
Two invitations arrive and the second looks blank	Gmail collapses a later message that resembles an earlier one in the same thread behind "Show quoted text".	Expand the quoted text. The three roles now have three different subject lines, which prevents most of this.
"already has a Hope Beacon account"	That address finished a sign-up before, possibly at another church.	If they are in your church, change their role from the member list. If they have genuinely left, delete the account, which frees the address.
The install button does nothing on an iPhone	Not Safari. Chrome, Firefox, Edge and in-app browsers cannot install on iOS.	Tap **Open this page in Safari** on the card, then Share, then Add to Home Screen. If that button does nothing, the browser refused the handoff: use its ••• menu instead.
The icon opens Safari with an address bar	What was added is a bookmark from before the fix.	Delete the icon and add it again from Safari.
"This copy can never update"	It was installed from a temporary preview address.	Open the real address, install from there, then delete the old icon.
A setting was changed and nothing happened	The two settings beginning `NEXT_PUBLIC_` are read when the site is built.	Redeploy. Saving alone changes nothing.
Everybody was signed out at once	The database project changed, the web address changed, or the project's signing secret was rotated. A code deploy does not do this.	See Part 7. If the address changed, people must reinstall as well.
"Your account is not ready. JWT	Fixed on 26 August 2026. A sign-in lasts one hour unless the app trades its refresh token for a	Nothing to do. A session now lasts until somebody signs out, and coming back to the app

What you see	What it usually is	What to do
expired."	new one, and nothing did, so everybody was signed out an hour after signing in.	re-checks it.
A stage was advanced by mistake	Advance is one tap.	**Undo, step back** beside it. Recorded as a correction, and the Explorer never sees their stage either way.
A Guide cannot take another Explorer	They are at the church's limit, five by default.	Pair with another Guide, recruit one, or raise the limit in Church settings. Do not raise it to solve a shortage of Guides.
The whole left column is invisible on a dark theme	Fixed on 26 August 2026. The page background was not being themed, so light text landed on a light page.	Nothing to do.
A Guide cannot be given another Explorer	They already have five. The database refuses a sixth.	Pair with a different Guide, or recruit one. Do not raise the cap to solve a shortage of Guides.

11. For an AI tool continuing this

Read this section before making a change. It states what is true, what must stay true, and the mistakes already made here so they are not made twice.

The shape of it

- Next.js App Router, TypeScript, Tailwind. Supabase for database and authentication. One edge function, `invite`, holding the only service role key.
- The app runs with **no backend at all**, on sample data in the browser. That is not a fallback, it is a supported mode with tests that fail if it breaks. Never write code that requires the database to exist.
- The security model lives in `supabase/migrations/`. Contracts evolve by adding a numbered migration, never by editing one that has already been applied.

Invariants you must not break

1. No screen may let anybody set their own role. There is no `setMyRole`, and saving a profile always writes back the role it read.
2. The service role key never reaches the browser and never appears in a variable named `NEXT_PUBLIC_*`.
3. Never mint an invitation link after sending one. An account has one slot and the second mint destroys the first.
4. Removing a member goes through `remove_member_by_leader`. Deleting the profile row alone leaves the account behind and locks the address out for good.
5. Nothing in the update path may clear the browser's stored session.
6. Text a member reads carries no em dashes, calls a Guide a Guide, and does not reach for the cadences a machine reaches for.

Prove it before you claim it

```
npm run verify        # 24 checks: types, build, security, copy, email, install
npm run build         # must pass before anything is pushed
node tests/plain-words.mjs
node tests/accounts-and-sessions.mjs
```

> **CAUTION** · A test that passes first time has proved nothing
>
> Every check here was written alongside a negative control: break the thing deliberately, watch the test fail, then restore it. Two checks in this repository passed cleanly over the exact bug they existed to catch, and only the negative control found that out. One reported "all OK" when it had not been able to look at anything at all.
>
> If you add a test, break the code and watch it fail before you believe it.

Mistakes already made here

- **Quoting a count without fetching first.** A stale local copy produced a number four times too large, and it reached a decision.
- **Presenting a blocked network as a design choice.** If something could not be done, say that plainly, then give the reasoning for the fallback separately.
- **Calling unverified work verified.** Pushing is not deploying, and deploying is not observing. Say which of the three happened.
- **Grepping the output of a script for "FAIL" only.** A script that crashed before printing anything looked exactly like a pass.

12. What is not finished

Stated plainly, because a plan that hides its gaps is worse than no plan.

Item	Status
Whether the latest deploy is live	Unverified from here. The sandbox cannot reach the site or the hosting dashboard. Needs a person with a browser.
The iPhone install fix on real Safari	The missing tag is confirmed in the built page. It has not been tested on a physical iPhone.
A restored backup	The job runs and its failure paths are tested. No restore has ever been performed.
Non-English wording	Eleven translations still use the old words for Explorer and Guide. Whether those names translate at all is a decision per language.
Two end-to-end tests	Failing before this work started, in the guided tutorial. Diagnose before demonstrating the tutorial.
Creating a new church without a developer	Possible in the database, not yet possible from a screen.
Bulk invitations by dropped spreadsheet	Pasting a list **is built**. Dragging a .csv or .xlsx onto the screen is not, and is the remaining stage.
The one-tap Safari handoff on a real device	Tested against simulated iPhone browsers. Never run on a physical iPhone.
Publishing a post, from a real sign-in	The rules were proved against the live database as a real Explorer, Guide, Director and Executive Director, including that a draft never reaches anybody else. It has not been done through the app by a person with a password.
"Active" as a count of visits	Not built and deliberately so. Beacon does not record when somebody opens the app, and the screen says what the number does mean instead of implying otherwise.

Bulk invitations, as specified

Inviting twenty-five people one form at a time is not a workflow, and it is the next thing to build. The design is in three stages so each can ship and be used on its own.

Stage	What it does	The rule it must not break
1. Paste a list	Paste any number of addresses, choose one role for the batch, see every row parsed in a preview, then send. Each row reports its own result.	Nothing is sent until the Director has seen the preview. Duplicates, malformed addresses and people who are already members are flagged before sending, not after.
2. Suggested pairing	After a batch of Explorers, propose which Guide takes whom, and show the whole proposal for approval.	The cap of five is respected, and nobody is ever paired silently. A pairing is a relationship between two people, not a row in a table.
3. Drop a file	Drag a spreadsheet onto the screen. The app finds the email, name and role columns and shows what it found.	Detection is always shown and always correctable. A mis-read role column would invite twenty-five people as Directors, and that is not a mistake you can take back.

> **NOTE** · The one sentence to keep
>
> The app's limit is not servers, it is Guides. Everything about running this well follows from that: recruit a Guide, train them, pair them with up to five people, and watch the number of Explorers waiting for one.